

Capítulo 6

MongoDB, Mongoose y operaciones CRUD

Desarrollo de Software IX · Código 1493 · Módulo III

Propósito del capítulo

Explicar modelado y persistencia de datos en MongoDB con Mongoose, incluyendo documentos, esquemas, validación, relaciones, consultas, operaciones CRUD, índices y datos semilla.

Tabla de contenido

1. Panorama del capítulo	3
2. Dos modelos de datos: relacional y documental	3
3. MongoDB: base de datos, colección, documento y BSON	4
4. Diseño de esquemas documentales: embeber o referenciar	5
5. Mongoose como capa de modelado (ODM)	6
6. Operaciones CRUD y su correspondencia con REST	7
7. Consultas: filtros, proyección, orden y paginación	8
8. Índices: qué aceleran y qué cuestan	8
9. Consistencia: atomicidad por documento y transacciones	9
10. Validación: capa de datos y capa de aplicación	9
11. Datos semilla	10
12. Operación con Docker: mongod en contenedor	10
13. Caso guía: sistema de reservas	11
14. Actividades sugeridas	11
15. Rúbrica breve	12
16. Cierre	12

1. Panorama del capítulo

La base de datos representa la memoria del sistema. Un buen modelo de datos permite que el backend responda con consistencia, que la interfaz muestre información útil y que las reglas de negocio puedan verificarse. MongoDB trabaja con documentos, lo que ofrece flexibilidad, pero esa flexibilidad requiere criterio. Este capítulo estudia el porqué de las decisiones de persistencia: cuándo conviene un modelo documental, cómo se diseña un esquema, qué problema resuelve un ODM como Mongoose, cómo se corresponden las operaciones CRUD con una API REST, y qué papel cumplen las consultas, los índices, la atomicidad y la operación con contenedores.

Idea central

Modelar datos no es copiar pantallas. Es identificar entidades del dominio, sus atributos, relaciones, restricciones y formas de consulta. La estructura de los datos debe responder a cómo la aplicación los lee y los escribe, no a cómo se ven en una maqueta.

2. Dos modelos de datos: relacional y documental

Antes de escribir un esquema conviene entender el panorama. Durante décadas, el modelo relacional dominó la persistencia: los datos se organizan en tablas con filas y columnas, cada tabla tiene un esquema fijo y las relaciones se expresan mediante claves foráneas. El modelo documental, popularizado por MongoDB, organiza los datos en documentos autocontenidos que se parecen a objetos JSON. Ninguno de los dos es superior en abstracto; cada uno optimiza supuestos distintos sobre cómo se consultan y evolucionan los datos.

El modelo relacional se apoya en la normalización: descomponer la información en tablas pequeñas sin redundancia, de modo que cada hecho se almacene una sola vez. La normalización protege la integridad —actualizar el nombre de un cliente toca una sola fila— y permite combinaciones arbitrarias mediante JOIN. Su costo es que reconstruir una entidad completa exige unir varias tablas, y que el esquema rígido obliga a migraciones formales cuando el dominio cambia.

El modelo documental invierte la apuesta: acepta cierta denormalización —duplicar datos pequeños dentro del documento que los usa— a cambio de que una lectura típica resuelva con un solo acceso. Un documento de pedido puede llevar dentro sus líneas de detalle y un resumen del cliente; leer el pedido no requiere unir nada. El costo es simétrico: si el dato duplicado cambia en su origen, la aplicación debe decidir si las copias se actualizan, se aceptan como foto histórica o se evitan usando referencias. La denormalización no es descuido; es una decisión consciente que intercambia integridad automática por rendimiento y simplicidad de lectura.

Aspecto	Modelo relacional	Modelo documental
Unidad de datos	Fila dentro de una tabla con esquema fijo.	Documento autocontenido, estructura flexible.
Relaciones	Claves foráneas y JOIN en el motor.	Embebido o referencias resueltas por la aplicación.
Integridad	Restricciones declarativas (FK, UNIQUE, CHECK).	Validación en la capa de aplicación u ODM.

Aspecto	Modelo relacional	Modelo documental
Evolución del esquema	Migraciones formales.	Documentos pueden diferir; la aplicación tolera versiones.
Caso favorable	Datos muy interconectados, reportes ad hoc, transacciones complejas.	Entidades que se leen completas, esquemas cambiantes, prototipos y agregados de dominio.

Criterio de elección

Pregunte cómo se leerán los datos. Si la mayoría de las pantallas necesita una entidad completa con sus partes, el documento autocontenido es natural. Si el valor está en cruzar entidades de formas imprevistas —reportes, análisis, combinaciones nuevas cada semana—, el modelo relacional conserva la ventaja. En un proyecto Full Stack académico con MongoDB, la lección es diseñar documentos pensando en las consultas, no trasladar tablas relacionales una a una.

3. MongoDB: base de datos, colección, documento y BSON

MongoDB organiza información en una jerarquía de tres niveles: una base de datos agrupa colecciones, una colección agrupa documentos y un documento contiene campos. A diferencia de una tabla, la colección no impone esquema: dos documentos de la misma colección pueden tener campos distintos. Esa libertad es útil durante la evolución del proyecto, pero traslada a la aplicación la responsabilidad de mantener consistencia, que es precisamente el espacio que ocupará Mongoose.

Concepto	Descripción	Ejemplo
Base de datos	Agrupa colecciones de una aplicación.	dsix_reservas
Colección	Agrupa documentos del mismo tipo lógico.	usuarios, reservas, productos
Documento	Registro individual en formato BSON.	{ nombre, correo, rol }
Campo	Dato dentro del documento.	estado, fecha, total
ObjectId	Identificador único generado por defecto.	_id: ObjectId(...)

Internamente, MongoDB no guarda JSON textual sino BSON (Binary JSON): una representación binaria que añade tipos ausentes en JSON —fechas reales, enteros de distinta precisión, decimales exactos, datos binarios y el propio ObjectId— y que puede recorrerse con eficiencia sin reinterpretar texto. Entender que existe BSON explica comportamientos que de otro modo parecen arbitrarios: por qué una fecha guardada se compara correctamente por rango, por qué `"10"` y ``10`` no coinciden en un filtro, y por qué el ``_id`` por defecto no es una cadena cualquiera sino un valor de 12 bytes que codifica marca de tiempo y aleatoriedad, lo que permite generarlo en el cliente sin coordinación central.

- JSON es el formato de intercambio con el cliente; BSON es el formato de almacenamiento y transporte interno.
- Los tipos importan: una fecha guardada como cadena no se ordena ni filtra como fecha.

- El ObjectId incluye marca de tiempo, por lo que ordenar por `\_id` aproxima el orden de creación.
- Un documento tiene límite de tamaño (16 MB), señal de diseño: los arreglos que crecen sin cota no deben embeberse.
- La ausencia de esquema en el motor no significa ausencia de esquema en la aplicación.

4. Diseño de esquemas documentales: embeber o referenciar

La decisión central del modelado documental es si un dato relacionado se embebe dentro del documento principal o se almacena en su propia colección y se referencia por identificador. Embebido funciona bien cuando la información se consulta junta, depende del documento padre y no crece indefinidamente. Referencia funciona mejor cuando la entidad tiene vida propia, se comparte entre varios documentos o acumula registros sin límite. La pregunta operativa es doble: ¿quién lee este dato y junto a qué?, y ¿quién lo actualiza y con qué frecuencia?

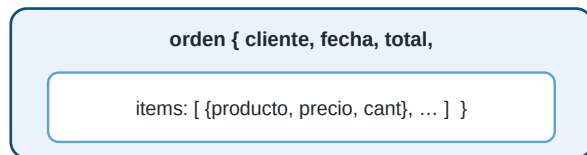
Existe además un patrón mixto, muy frecuente en sistemas reales: embeber un resumen (los dos o tres campos que la pantalla necesita siempre) y referenciar el documento completo. Un pedido guarda el nombre y el precio del producto al momento de la compra —foto histórica que no debe cambiar aunque el catálogo cambie— y además el `ObjectId` del producto para consultar su detalle vigente. El patrón mixto convierte la duplicación en una decisión semántica: qué datos son históricos y qué datos son vivos.

Criterio	Favorece embeber	Favorece referenciar
Patrón de lectura	Siempre se lee junto al padre.	Se consulta de forma independiente.
Cardinalidad	Pocos elementos, cota conocida.	Crece sin límite (logs, historial).
Propiedad	Solo existe dentro del padre.	Entidad compartida por varios documentos.
Actualización	Cambia junto con el padre o nunca.	Cambia con frecuencia en su origen.
Atomicidad	Se requiere actualizar todo junto.	Las partes pueden actualizarse por separado.

```
// Patrón mixto: resumen embebido + referencia al documento vivo
const pedidoSchema = new mongoose.Schema({
  usuario: { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario', required: true },
  items: [{
    producto: { type: mongoose.Schema.Types.ObjectId, ref: 'Producto' },
    nombre: String, // foto histórica al momento de la compra
    precio: Number, // no cambia aunque el catálogo cambie
    cantidad: { type: Number, min: 1 },
  }],
  total: Number,
}, { timestamps: true });
```

Regla práctica

Embeba lo que es parte del documento; referencie lo que es protagonista de su propia historia. Las direcciones de un usuario rara vez justifican una colección propia; el usuario que crea una reserva siempre la justifica.

Embeber: el todo y sus partes viajan juntos

una sola lectura · atomicidad por documento

Referenciar: cada entidad vive aparte

sin duplicación · el stock se actualiza en un solo lugar

Criterio: lo que se lee junto y pertenece al padre se embebe; lo compartido y cambiante se referencia.

5. Mongoose como capa de modelado (ODM)

Mongoose es un ODM (Object Document Mapper): un puente entre los objetos de JavaScript y los documentos de MongoDB. El problema que resuelve es concreto. MongoDB no exige esquema, pero una aplicación sí necesita uno: necesita garantizar que todo usuario tenga correo, que el estado de una reserva pertenezca a un conjunto cerrado, que las fechas sean fechas. Sin ODM, cada controlador tendría que repetir esas comprobaciones. Mongoose las centraliza en cuatro aportes: esquemas que documentan la forma esperada de cada colección, casting que convierte tipos automáticamente (la cadena "2026-06-15" se vuelve `Date`), validación declarativa que rechaza documentos inválidos antes de tocar la base, y middleware que ejecuta lógica en momentos clave del ciclo de vida del documento.

- Definir tipos y campos requeridos en el esquema, reflejando reglas del dominio y no nombres arbitrarios.
- Usar `enum` para estados controlados del flujo de negocio.
- Agregar valores por defecto cuando tenga sentido (estado inicial, fecha de creación).
- Ocultar campos sensibles al serializar (por ejemplo, el hash de contraseña).
- Centralizar en el esquema las validaciones que pertenecen a los datos, no a un caso de uso particular.

El esquema es la pieza declarativa: describe campos, tipos, restricciones y opciones. Del esquema se compila un modelo, que es la clase con la que el código crea, consulta y modifica documentos. Cuando un dato llega del cliente, Mongoose intenta el casting al tipo declarado; si falla, o si una validación no se cumple, la operación se rechaza con un error descriptivo que el backend puede traducir a una respuesta 400. Esta barrera convierte al esquema en un contrato verificable entre la API y la base de datos.

```

const usuarioSchema = new mongoose.Schema({
  nombre: { type: String, required: true, trim: true },
  correo: {
    type: String, required: true, unique: true,
    lowercase: true, match: /.+@.+\./,
  },
  rol: { type: String, enum: ['admin', 'estudiante'], default: 'estudiante' },
  passwordHash: { type: String, required: true, select: false },
}, { timestamps: true });

const Usuario = mongoose.model('Usuario', usuarioSchema);

```

El middleware de modelo (hooks) permite interceptar el ciclo de vida de un documento: antes de guardar (`pre('save')`), después de guardar, antes de eliminar. Su valor teórico es la cohesión: la regla "toda contraseña se guarda con hash" vive junto al esquema de usuario, de modo que ningún controlador puede olvidarla. El middleware no debe convertirse en un cajón de lógica de negocio arbitraria; su lugar son las invariantes del dato, aquellas que deben cumplirse sin importar desde dónde se escriba.

```
usuarioSchema.pre('save', async function () {
  if (this.isModified('passwordHash')) {
    this.passwordHash = await bcrypt.hash(this.passwordHash, 10);
  }
});

usuarioSchema.methods.compararPassword = function (plano) {
  return bcrypt.compare(plano, this.passwordHash);
};
```

6. Operaciones CRUD y su correspondencia con REST

CRUD resume las cuatro operaciones básicas sobre datos persistentes: crear, leer, actualizar y eliminar. En una API REST estas operaciones se expresan mediante métodos HTTP sobre recursos, y la correspondencia no es casual: REST modela el estado de recursos y CRUD modela su manipulación, de modo que cada verbo HTTP encuentra su operación natural en la base de datos. Mantener esta correspondencia hace la API predecible: quien conoce el recurso puede inferir las rutas, y quien lee una ruta puede inferir su efecto.

Método HTTP	Operación	Mongoose	Ruta típica
POST	Crear	<code>Model.create(data)</code>	<code>POST /api/reservas</code>
GET	Leer varios	<code>Model.find(filtro)</code>	<code>GET /api/reservas</code>
GET	Leer uno	<code>Model.findById(id)</code>	<code>GET /api/reservas/:id</code>
PUT / PATCH	Actualizar	<code>findByIdAndUpdate(id, data)</code>	<code>PUT /api/reservas/:id</code>
DELETE	Eliminar	<code>findByIdAndDelete(id)</code> o baja lógica	<code>DELETE /api/reservas/:id</code>

```
// Lectura de un recurso con manejo explícito de ausencia
app.get('/api/reservas/:id', async (req, res) => {
  const reserva = await Reserva.findById(req.params.id)
    .populate('usuario', 'nombre correo');
  if (!reserva) {
    return res.status(404).json({ error: 'Reserva no encontrada' });
  }
  res.json(reserva);
});
```

PUT, PATCH y la baja lógica

PUT expresa reemplazo completo del recurso y PATCH una modificación parcial; ambos deben ser idempotentes en su intención. En cuanto a DELETE, muchos dominios prefieren la baja lógica —marcar `activo: false`— porque conserva historial y referencias. Eliminar físicamente un usuario que tiene reservas asociadas rompe la trazabilidad del sistema.

7. Consultas: filtros, proyección, orden y paginación

Un proyecto no solo guarda datos; sobre todo los consulta. Desde la formulación deben identificarse las consultas principales: por usuario, por estado, por fecha, por categoría. Una consulta bien construida combina cuatro elementos: el filtro (qué documentos califican), la proyección (qué campos se devuelven), el orden (cómo se presentan) y la paginación (cuántos por vez). La proyección merece énfasis: devolver documentos completos cuando la pantalla necesita tres campos desperdicia red y expone datos innecesarios; pedir solo lo que se usa es a la vez una práctica de rendimiento y de seguridad.

```
// filtro + proyección + orden + paginación
const pagina = Number(req.query.page) || 1;
const porPagina = 10;

const reservas = await Reserva
  .find({ estado: 'pendiente', fecha: { $gte: hoy } }) // filtro
  .select('fecha bloque estado usuario')              // proyección
  .sort({ fecha: 1, _id: 1 })                          // orden estable
  .skip((pagina - 1) * porPagina)                     // paginación
  .limit(porPagina);
```

¿Por qué paginar? Porque las colecciones crecen y las pantallas no. Una lista que hoy devuelve 30 documentos devolverá 30 000 en producción: sin paginación, cada petición transferirá megabytes, el servidor retendrá memoria proporcional al tamaño de la colección y la interfaz tardará en renderizar elementos que nadie verá. Pagar acota el costo de cada petición a una constante, independiente del tamaño total de los datos. El orden, por su parte, debe ser estable: si dos documentos empatan en el campo de orden, agregar `\_id` como desempate garantiza que las páginas no se solapen entre peticiones.

8. Índices: qué aceleran y qué cuestan

Sin índice, MongoDB resuelve un filtro recorriendo la colección completa (collection scan): el costo crece linealmente con el número de documentos. Un índice es una estructura ordenada auxiliar —conceptualmente, un árbol balanceado sobre uno o más campos— que permite saltar directamente a los documentos que califican, como el índice alfabético de un libro evita leerlo entero. La consulta por correo que tardaba proporcional al millón de usuarios pasa a costar proporcional a la profundidad del árbol. Pero los índices no son gratuitos, y entender su costo evita el reflejo de indexarlo todo.

Dimensión	Lo que el índice acelera	Lo que el índice cuesta
Lectura	Filtros y órdenes sobre los campos indexados.	Nada significativo: la lectura solo gana.
Escritura	—	Cada insert/update debe actualizar también cada índice.
Almacenamiento	—	Espacio adicional en disco y en memoria.
Mantenimiento	Consultas frecuentes predecibles.	Índices sin uso que penalizan escrituras a cambio de nada.

Criterio de indexación

Indexe los campos que aparecen en los filtros y órdenes de las consultas frecuentes de su aplicación —y solo esos—. El índice ``unique`` cumple doble función: acelera la búsqueda y garantiza unicidad a nivel de motor, algo que la validación de aplicación por sí sola no puede asegurar frente a peticiones concurrentes.

9. Consistencia: atomicidad por documento y transacciones

En MongoDB, toda escritura sobre un único documento es atómica: si una actualización modifica cinco campos y un arreglo embebido, ningún lector verá un estado intermedio con solo parte de los cambios aplicados. Esta garantía explica una consecuencia profunda del modelado documental: embeber no es solo una optimización de lectura, es una decisión de consistencia. Si el pedido y sus líneas viven en el mismo documento, actualizar el total y agregar una línea ocurre como una sola operación indivisible. Si vivieran en colecciones separadas, una falla entre ambas escrituras dejaría datos incoherentes.

Para los casos donde la operación debe abarcar varios documentos —descontar inventario y crear el pedido, transferir entre dos cuentas—, MongoDB ofrece transacciones multi-documento. Funcionan, pero tienen costo en complejidad y rendimiento, y requieren configuración de replica set. La lección de diseño es de orden: primero intente que las invariantes críticas quepan en un documento, aprovechando la atomicidad natural; recurra a transacciones solo cuando el dominio genuinamente exige coordinar entidades independientes. Un esquema que necesita transacciones para cada operación cotidiana suele estar delatando un modelado relacional disfrazado de documental.

Principio de diseño

La unidad de atomicidad es el documento. Diseñe los documentos de modo que cada operación de negocio frecuente modifique uno solo; reserve las transacciones para las operaciones excepcionales que cruzan entidades.

10. Validación: capa de datos y capa de aplicación

La base de datos no debe recibir cualquier cosa, pero ¿dónde se valida? La respuesta correcta es: en varias capas, con propósitos distintos. La validación del frontend mejora la usabilidad —retroalimentación inmediata— pero no protege nada, porque cualquier cliente puede saltarla. La validación de la capa de aplicación (en rutas o controladores, antes de tocar el modelo) produce errores claros y específicos del caso de uso. La validación del esquema Mongoose es la red final: garantiza las invariantes del dato sin importar desde qué controlador, script o seed se escriba. Las reglas estructurales pertenecen al esquema; las reglas contextuales ("solo un admin puede aprobar") pertenecen a la aplicación.

Regla	Ejemplo	Motivo
required	correo obligatorio	Evita documentos incompletos.
unique (índice)	correo único	Evita duplicados incluso bajo concurrencia.
enum	estado: pendiente/aprobado/rechazado	Controla el flujo del dominio.

Regla	Ejemplo	Motivo
min/max	cantidad mayor que cero	Evita valores absurdos.
trim/lowercase	correo normalizado	Reduce inconsistencias de entrada.
match	formato de correo	Rechaza estructuras inválidas temprano.

Una advertencia técnica con valor conceptual: ``unique`` en Mongoose no es una validación sino la declaración de un índice único. La diferencia importa porque la unicidad real solo puede garantizarla el motor: dos peticiones simultáneas con el mismo correo pasarían ambas una verificación hecha en la aplicación (leer y luego escribir), pero el índice único rechazará la segunda escritura. Es el ejemplo canónico de por qué ciertas reglas deben vivir en la capa de datos.

11. Datos semilla

Los datos semilla (seed) permiten probar el sistema sin depender de capturas manuales. Un buen proyecto incluye un script reproducible que limpia las colecciones de desarrollo y las puebla con usuarios de prueba, entidades principales y casos que representen estados distintos del flujo: una reserva pendiente, una aprobada, una rechazada. El valor es metodológico: cualquier integrante del equipo —o quien evalúe el sistema— puede levantar el proyecto y ejercitar todos los caminos sin construir el escenario a mano. El seed debe pasar por los modelos de Mongoose, no insertar directamente, para que las validaciones y el middleware (como el hash de contraseñas) se apliquen también a los datos de prueba.

Dato semilla mínimo

Incluya al menos un usuario administrador, dos usuarios normales y registros en diferentes estados. El demo debe poder ejecutarse sin crear todo desde cero.

12. Operación con Docker: mongod en contenedor

En desarrollo moderno, MongoDB rara vez se instala directamente sobre el sistema operativo: se ejecuta como contenedor. El proceso ``mongod`` corre dentro de una imagen oficial, expone su puerto (27017) hacia la máquina anfitriona y todo el equipo trabaja contra la misma versión exacta del motor, sin conflictos con otras instalaciones. El contenedor encapsula el servidor de base de datos como una dependencia declarada del proyecto, igual que ``package.json`` declara las dependencias de Node.

El concepto clave es la persistencia mediante volúmenes. El sistema de archivos de un contenedor es efímero: si el contenedor se elimina, sus datos desaparecen con él. Un volumen es un área de almacenamiento gestionada por Docker que vive fuera del ciclo de vida del contenedor y se monta en la ruta donde `mongod` escribe sus datos (``/data/db``). Con el volumen, destruir y recrear el contenedor —actualizar la versión, reiniciar el entorno— no pierde las colecciones: el contenedor es desechable, los datos no. Separar cómputo efímero de estado persistente es uno de los principios centrales de la operación con contenedores.

```
# docker-compose.yml (fragmento)
services:
  mongo:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db    # persistencia fuera del contenedor

volumes:
  mongo_data:

# La aplicación se conecta vía variable de entorno:
# MONGODB_URI=mongodb://localhost:27017/dsix_reservas
```

13. Caso guía: sistema de reservas

El caso guía integra las decisiones del capítulo. El dominio tiene cuatro entidades: usuarios y laboratorios son entidades independientes y referenciadas; las reservas referencian a ambas porque cada una tiene vida propia y consultas propias; los comentarios de seguimiento, en cambio, podrían embeberse en la reserva si su número es acotado, o referenciarse si el historial crece sin límite — exactamente la tabla de decisión de la sección 4 aplicada a un caso concreto. Obsérvese además, en el esquema siguiente, el índice único compuesto: la regla de negocio "no puede haber dos reservas del mismo laboratorio en el mismo bloque y fecha" se garantiza en la capa de datos, donde ninguna condición de carrera puede violarla.

Colección	Campos mínimos	Consultas esperadas
usuarios	nombre, correo, passwordHash, rol	login, perfil, administración.
laboratorios	nombre, ubicación, capacidad, activo	listar disponibles, filtrar activos.
reservas	usuario, laboratorio, fecha, bloque, estado	por usuario, por estado, por fecha.
comentarios	reserva, autor, texto, fecha	historial de cambios o seguimiento.

```
const reservaSchema = new mongoose.Schema({
  usuario: { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario', required: true },
  laboratorio: { type: mongoose.Schema.Types.ObjectId, ref: 'Laboratorio', required: true },
  fecha: { type: Date, required: true },
  bloque: { type: String, enum: ['8-10', '10-12', '14-16'], required: true },
  estado: {
    type: String,
    enum: ['pendiente', 'aprobada', 'rechazada'],
    default: 'pendiente',
  },
}, { timestamps: true });

// Una reserva por usuario, laboratorio, fecha y bloque
reservaSchema.index(
  { laboratorio: 1, fecha: 1, bloque: 1 },
  { unique: true }
);
```

14. Actividades sugeridas

- Identificar las entidades principales del proyecto y sus relaciones.
- Decidir, con la tabla de criterios de la sección 4, qué datos se embeben y cuáles se referencian, justificando cada caso.
- Crear un esquema Mongoose para una entidad central con tipos, requeridos, enum y valores por defecto.
- Definir cinco validaciones de datos y clasificar cada una como estructural (esquema) o contextual (aplicación).
- Escribir la consulta principal de la aplicación con filtro, proyección, orden estable y paginación.
- Identificar la operación de negocio más crítica y verificar si cabe en la atomicidad de un solo documento.
- Levantar MongoDB en contenedor con volumen y comprobar que los datos sobreviven a recrear el contenedor.
- Diseñar datos semilla que cubran todos los estados del flujo principal.

15. Rúbrica breve

Criterio	Excelente	Satisfactorio	Insuficiente
Modelado	Decisiones embeber/referenciar justificadas por lecturas y atomicidad.	Esquemas correctos con justificación parcial.	Tablas relacionales copiadas como documentos sin criterio.
Esquemas Mongoose	Tipos, validaciones, enum y middleware coherentes con el dominio.	Esquemas funcionales con validación básica.	Campos sin tipos ni reglas; validación ausente.
CRUD y consultas	Correspondencia REST clara; consultas con proyección y paginación.	CRUD completo con consultas simples.	Rutas inconsistentes; listas sin paginar.
Operación	Mongo en contenedor con volumen y seed reproducible.	Base operativa con seed parcial.	Entorno manual sin persistencia ni datos de prueba.

16. Cierre

MongoDB aporta flexibilidad, pero el proyecto necesita estructura. El recorrido del capítulo va de la teoría a la práctica: elegir el modelo documental con conciencia de lo que se gana y se cede frente al relacional; diseñar documentos a partir de las lecturas del dominio y de la atomicidad que requieren sus operaciones; usar Mongoose para convertir reglas de negocio en esquemas, validaciones y middleware verificables; expresar el CRUD con la gramática de REST; consultar con proyección, orden y paginación; indexar con intención; y operar el motor como un contenedor cuyo estado vive en volúmenes. Un modelo de datos pensado así reduce errores en el backend, simplifica la interfaz y deja huellas claras de las decisiones que lo produjeron.